

Rabin-Karp Algorithm for Pattern Searching

Last Updated : 01 Aug, 2025



In many data structures and algorithms (DSA) problems, one common task is to compare strings—whether it's finding a word in a sentence, detecting duplicates, or checking for patterns inside a larger text.

Think of this like trying to find a short phrase inside a big paragraph—manually checking every character would be tiring. Computers do the same thing, and if we're not careful, it can become very slow for long texts.

That's where the Rabin-Karp algorithm comes in.

It was developed by Michael Rabin and Richard Karp in 1987 as a clever way to speed up pattern matching using hashing techniques. Instead of comparing characters one by one, it converts the strings into numbers (hashes) and compares those—just like how barcodes are used in stores instead of reading the full product name.

In short, Rabin-Karp is:

- A string matching algorithm
- Uses rolling hash to find all occurrences of a pattern in a text
- Optimized for cases where multiple patterns or repeated matching is required

This forms the foundation for many modern algorithms in plagiarism detection, substring search, and more.

Table of Content

- [Why is Rabin-Karp Needed?](#)
- [Key Idea of Rabin-Karp](#)
- [Practical Example for Better Understanding](#)
- [Limitations of Rabin-Karp](#)
- [Need for Double Hashing](#)
- [Types of Problems Rabin-Karp Can Solve](#)

Why is Rabin-Karp Needed?

Let's say you want to find if the word "code" appears inside a big article.

The simplest way? Start from the first letter and check every group of 4 letters one by one. This is known as the naive approach, and it works fine—but it checks every character every time, which can be very slow if the text is long.

The naive method has a time complexity of $O(n \times m)$, where:

- n = length of the text
- m = length of the pattern

Now imagine trying to find multiple patterns (like checking 100 different words in a document). Doing this the naive way becomes painfully slow.

Rabin-Karp improves this by using rolling hash.

Instead of comparing actual characters every time, it converts both the pattern and parts of the text into numbers (hashes) and compares those numbers. This way, most of the time it can detect mismatches without looking at every character, making it much faster and scalable.

Key Idea of Rabin-Karp

The main idea behind Rabin-Karp is to convert strings into numeric hashes so we can compare numbers instead of characters. This makes the process much faster. To do this efficiently, we use a **rolling hash**, which allows us to compute the hash of the next substring in constant time by updating the previous hash. So instead of rechecking every character, we just slide the window and adjust the hash.

We first compute the hash of the pattern, then compare it with the hashes of all substrings of the text. If the hashes match, we do a quick character-by-character check to confirm (to avoid errors due to hash collisions).

This way, most comparisons are done using numbers, making pattern matching fast and scalable, especially for large texts or multiple patterns.

How is Hash Value calculated in Rabin-Karp?

The hash value in Rabin-Karp is calculated using a rolling hash function, which allows efficient hash updates as the pattern slides over the text. Instead of recalculating the entire hash for each substring, the rolling hash lets us remove the contribution of the old character and add the new one in constant time.

A string is converted into a numeric hash using a polynomial rolling hash. For a string s of length n , the hash is computed as:

$$\Rightarrow \text{hash}(s) = (s[0] * p^{(n-1)} + s[1] * p^{(n-2)} + \dots + s[n-1] * p^0) \% \text{mod}$$

Where:

- $s[i]$ is the numeric value of the i -th character ('a' = 1, 'b' = 2, ..., 'z' = 26)
- p is a small prime number (commonly 31 or 37)
- mod is a large prime number (like $1e9 + 7$) to avoid overflow and reduce hash collisions

This approach allows us to compute hash values of substrings in constant time using precomputed powers and prefix hashes.

Hash Recurrence Relation

Let $\text{preHash}[i]$ represent the hash of the prefix substring $s[0\dots i]$.

Then the recurrence is: **$\text{preHash}[i] = \text{preHash}[i - 1] * \text{base} + s[i]$**

Where:

- $p[0] = s[0]$
- $s[i]$ is the numeric value of the i -th character ('a' = 1, 'b' = 2, ..., 'z' = 26)
- base is a chosen prime number (commonly 31 or 37)
- All operations are done under modulo mod to avoid overflow

How to Calculate Hash from L to R ?

To get the hash of any substring $s[L\dots R]$ efficiently, we use prefix hashing.

First, we precompute prefix hashes, where $\text{prefix}[i]$ stores the hash of the substring $s[0\dots i]$. Alongside, we also precompute powers of the base (like base^1 , base^2 , etc.) to help in quick calculations.

Then, using modular arithmetic, we can calculate the hash of any range in constant time with the formula:

$$\text{hash}(L, R) = (\text{prefix}[R] - \text{prefix}[L-1] * \text{power}(R - L + 1) + \text{mod}) \% \text{mod}$$

Here,

- `prefix[R]` is the hash from 0 to R
- `prefix[L-1] * power(R - L + 1)` removes the contribution of the first L characters
- `+ mod` ensures the result stays positive before applying `% mod`

C++

Java

Python

C#

JavaScript

```
class RabinKarpHash {
private:
    const int mod = 1e9 + 7;
    const int base = 31;
    vector<int> hash;
    vector<int> power;

    // modular addition
    int add(int a, int b) {
        a += b;
        if (a >= mod) a -= mod;
        return a;
    }

    // modular subtraction
    int sub(int a, int b) {
        a -= b;
        if (a < 0) a += mod;
        return a;
    }

    // modular multiplication
    int mul(int a, int b) {
        return (int)((1LL * a * b) % mod);
    }

    // convert character to int
    // ('a' = 1, ..., 'z' = 26)
    int charToInt(char c) {
        return c - 'a' + 1;
    }
}
```

```

    }

public:
    // constructor: precomputes prefix hashes and powers
    RabinKarpHash(string &s) {
        int n = s.size();
        hash.resize(n);
        power.resize(n);

        hash[0] = charToInt(s[0]);
        power[0] = 1;

        for (int i = 1; i < n; ++i) {
            hash[i] = add(mul(hash[i - 1], base),
charToInt(s[i]));
            power[i] = mul(power[i - 1], base);
        }
    }

    // get hash of substring s[l...r] in O(1)
    int getSubHash(int l, int r) {
        int h = hash[r];
        if (l > 0) {
            h = sub(h, mul(hash[l - 1], power[r - l + 1]));
        }
        return h;
    }
};

```

Time Complexity: $O(n)$, the prefix hashes and powers are precomputed in a single pass over the string, which takes linear time. Once built, any substring hash can be retrieved in $O(1)$ time.

Auxiliary Space: $O(n)$, two arrays of size n are used to store the prefix hashes and powers of the base, resulting in linear extra space usage.

Practical Example for Better Understanding

Given two strings **text** (the text) and **pattern** (the pattern), consisting of lowercase English alphabets, find all 0-based starting indices where pattern occurs as a substring in text.

Examples:

Input: text = "geeksforgeeks", pattern = "geeks"

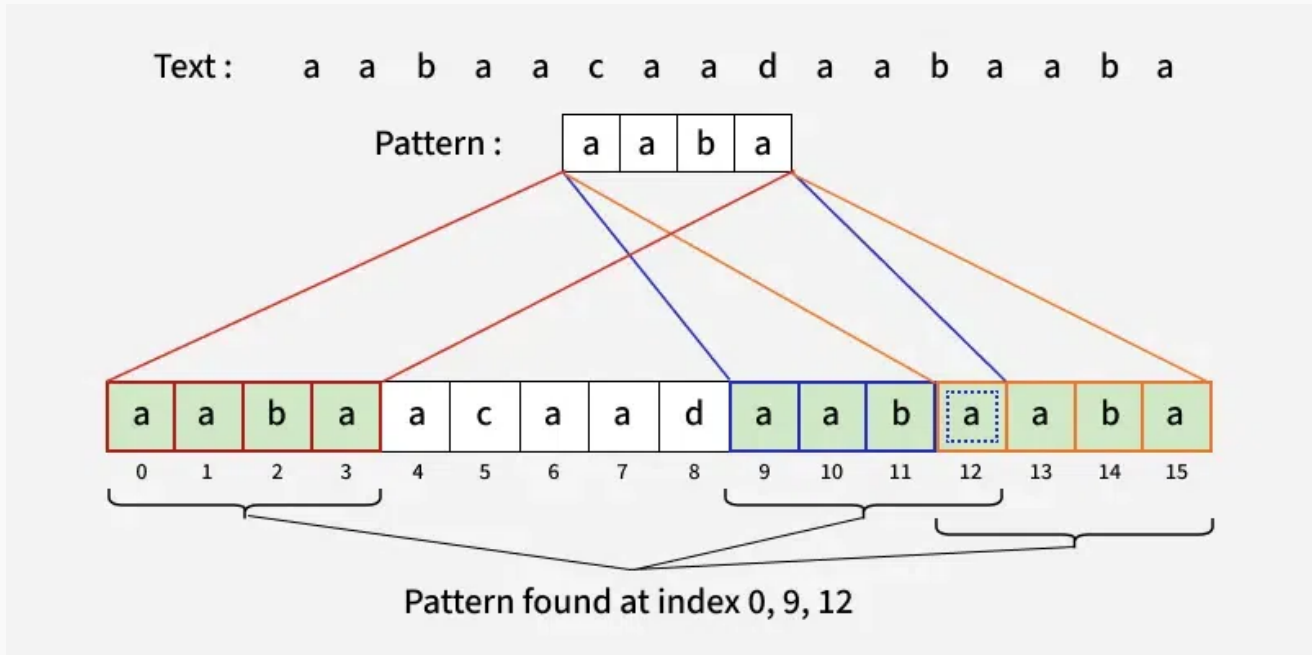
Output: [0, 8]

Explanation: The string "geeks" occurs at index 0 and 8 in text.

Input: text = "aabaacaadaabaaba", pattern = "aaba"

Output: [0, 9, 12]

Explanation:



In the [Naive String Matching](#) algorithm, we check whether every substring of the text of the pattern's size is equal to the pattern or not one by one.

Like the Naive Algorithm, the Rabin-Karp algorithm also check every substring. But unlike the Naive algorithm, the Rabin Karp algorithm matches the hash value of the pattern with the hash value of the current substring of text, and if the hash values match then only it starts matching individual characters. So Rabin Karp algorithm needs to calculate hash values for the following strings.

- Pattern itself
- All the substrings of the text of length m which is the size of pattern.

C++

Java

Python

C#

JavaScript

```
#include <iostream>
#include <vector>
```



```

#include <string>
using namespace std;

class RabinKarpHash {
private:
    const int mod = 1e9 + 7;
    const int base = 31;
    vector<int> hash;
    vector<int> power;

    // modular addition
    int add(int a, int b) {
        a += b;
        if (a >= mod) a -= mod;
        return a;
    }

    // modular subtraction
    int sub(int a, int b) {
        a -= b;
        if (a < 0) a += mod;
        return a;
    }

    // modular multiplication
    int mul(int a, int b) {
        return (int)((1LL * a * b) % mod);
    }

    // convert character to int
    // ('a' = 1, ..., 'z' = 26)
    int charToInt(char c) {
        return c - 'a' + 1;
    }

public:
    // constructor: precomputes prefix hashes and powers
    RabinKarpHash(string &s) {
        int n = s.size();
        hash.resize(n);
        power.resize(n);
    }

```

```

    hash[0] = charToInt(s[0]);
    power[0] = 1;

    for (int i = 1; i < n; ++i) {
        hash[i] = add(mul(hash[i - 1], base),
charToInt(s[i]));
        power[i] = mul(power[i - 1], base);
    }
}

// get hash of substring s[l...r] in O(1)
int getSubHash(int l, int r) {
    int h = hash[r];
    if (l > 0) {
        h = sub(h, mul(hash[l - 1], power[r - l + 1]));
    }
    return h;
}
};

// Rabin-Karp search using hash class
vector<int> searchPattern(string &text, string &pattern) {
    int n = text.size(), m = pattern.size();

    RabinKarpHash textHash(text);
    RabinKarpHash patHash(pattern);

    int patternHash = patHash.getSubHash(0, m - 1);
    vector<int> result;

    for (int i = 0; i <= n - m; i++) {
        int subHash = textHash.getSubHash(i, i + m - 1);
        if (subHash == patternHash) {
            result.push_back(i);
        }
    }

    return result;
}

int main() {
    string txt = "geeksforgeeks";

```



```

string pat = "geek";

vector<int> positions = searchPattern(txt, pat);
for (int idx : positions) {
    cout << idx << " ";
}
cout << endl;

return 0;
}

```

Output

0 8

Time Complexity: $O(n + m)$, we compute prefix hashes and powers for both text and pattern in $O(n + m)$. Then, we slide a window over the text, and each substring hash is compared in $O(1)$.

Auxiliary Space: $O(n + m)$, we store prefix hashes and power arrays for both text and pattern, taking $O(n + m)$ space.

Limitations of Rabin-Karp

- Hash collisions may occur — different substrings can have the same hash.
- Requires character-by-character check to confirm matches.
- Risk of modulo overflow if not handled with care.
- Performance depends on good hash function and prime modulus.
- Slightly higher constant factors compared to simpler algorithms like KMP.

Why Single Hash Can Fail (Hash Collisions):

When using a single hash function, there's always a chance that two different substrings may produce the same hash value. This is called a [hash collision](#).

Why does it happen?

=> We're computing hashes modulo a large number (e.g., $10^9 + 7$)

=> But since the number of possible substrings is huge, different substrings might accidentally result in the same hash after taking the modulo.

Consequence: If two different substrings have the same hash, the algorithm may falsely report a match (false positive). Rabin-Karp, in such a case, needs to verify the actual characters to confirm a match — which slows down performance.

Need for Double Hashing

To reduce the probability of collisions, we use double hashing — i.e., compute two independent hashes with different: Base values (p_1, p_2) and Moduli (mod_1, mod_2)

How it helps:

=> Now, two substrings are considered equal only if both hash values match.

=> The probability of two different substrings colliding in both hash functions is extremely low — roughly $1/(mod_1 \times mod_2)$, which is practically negligible.

C++ Java Python C# JavaScript

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

class RabinKarpHash {
private:
    const int mod1 = 1e9 + 7;
    const int mod2 = 1e9 + 9;
    const int base1 = 31;
    const int base2 = 37;

    vector<int> hash1, hash2;
    vector<int> power1, power2;

    // modular addition
    int add(int a, int b, int mod) {
        a += b;
        if (a >= mod) a -= mod;
        return a;
    }

    // modular subtraction
    int sub(int a, int b, int mod) {
```

```

        a -= b;
        if (a < 0) a += mod;
        return a;
    }

    // modular multiplication
    int mul(int a, int b, int mod) {
        return (int)((1LL * a * b) % mod);
    }

    // convert character to int
    int charToInt(char c) {
        return c - 'a' + 1;
    }

public:
    // constructor: precomputes both prefix hashes and powers
    RabinKarpDoubleHash(string &s) {
        int n = s.size();
        hash1.resize(n);
        hash2.resize(n);
        power1.resize(n);
        power2.resize(n);

        hash1[0] = charToInt(s[0]);
        hash2[0] = charToInt(s[0]);
        power1[0] = 1;
        power2[0] = 1;

        for (int i = 1; i < n; ++i) {
            hash1[i] = add(mul(hash1[i - 1], base1, mod1),
                           charToInt(s[i]),
mod1);

            power1[i] = mul(power1[i - 1], base1, mod1);

            hash2[i] = add(mul(hash2[i - 1], base2, mod2),
                           charToInt(s[i]),
mod2);

            power2[i] = mul(power2[i - 1], base2, mod2);
        }
    }

```

```

}

// get double hash of substring s[l...r]
vector<int> getSubHash(int l, int r) {
    int h1 = hash1[r];
    int h2 = hash2[r];
    if (l > 0) {
        h1 = sub(h1, mul(hash1[l - 1], power1[r - l + 1],
mod1), mod1);
        h2 = sub(h2, mul(hash2[l - 1], power2[r - l + 1],
mod2), mod2);
    }
    return {h1, h2};
}
};

```

Time Complexity: $O(n)$, the prefix hashes and powers are precomputed in a single pass over the string, which takes linear time. Once built, any substring hash can be retrieved in $O(1)$ time.

Auxiliary Space: $O(n)$, two arrays of size n are used to store the prefix hashes and powers of the base, resulting in linear extra space usage.

Types of Problems Rabin-Karp Can Solve

- Pattern matching – Find all occurrences of a pattern in a large text
- Plagiarism detection – Compare documents by checking for common substrings
- Multiple pattern search – Efficiently search for several patterns at once
- Substring comparison – Quickly compare substrings using hashes
- Palindrome and DP problems – Used for hashing-based optimizations
- Detect duplicate substrings – Find repeated sequences in strings
- Longest common prefix/suffix – In constant time using precomputed hashes

Search Pattern (Rabin-Karp Algorithm) | DSA Problem

 Comment

 K

kartik

+ Follow

 250



Article Tags :

Pattern Searching

DSA

Modular Arithmetic

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information